Carleton University

Department of Systems and Computer Engineering

Health and Fitness Club Management System

Project Report

Submitted by

Oscar Oguledo

Course: EGEN5208W — Databases for Software Engineers

Instructor: Abdelghny Orogat

Submission: April 2026

Table of Contents

1. Project Overview

1.1 Objective

This project designs and implements a comprehensive Health and Fitness Club Management System using PostgreSQL as the backend database. The system demonstrates sound database design principles, correct SQL usage, constraint enforcement, and role-based access control across three distinct user types. The full application stack is containerised using Docker and Docker Compose to ensure a consistent and reproducible runtime environment.

1.2 System Scope

The system manages three primary stakeholder categories, each with dedicated data entities and operations:

- Members — Client profiles, health metrics, and personal fitness goals.
- Trainers — Staff availability, session schedules, and assigned training sessions.
- Administrative Staff — Class management, room booking, and equipment maintenance.

1.3 Technology Stack

Component	Technology / Version
Backend	FastAPI (Python 3.9+)
Database	PostgreSQL 13+
ORM	SQLAlchemy 2.0 (async)
Frontend	React + TypeScript
Authentication	JSON Web Tokens (JWT) with bcrypt hashing
Containerisation	Docker & Docker Compose

2. Entity-Relationship Model

2.1 Overview

The ER diagram, submitted separately as ER_Diagram.pdf, represents the conceptual design of the system using Chen notation. It comprises 15 entities and more than 15 relationships, including two many-to-many associations resolved through dedicated junction tables.

2.2 Core Entities

Entity	Primary Key	Key Attributes	Purpose
Users	id (UUID)	email, password, role	Base authentication layer
Members	id (FK → Users)	full_name, dob, gender, phone	Member profile data
Trainers	id (FK → Users)	full_name	Trainer profile data
AdminStaff	id (FK → Users)	full_name	Administrator profile data
Rooms	id (UUID)	name, capacity	Physical club spaces
Equipments	id (UUID)	equipment_name, status, notes	Equipment tracking

2.3 Scheduling Entities

Entity	Primary Key	Key Attributes	Purpose
Classes	id (UUID)	name, trainer_id, room_id, date, times, capacity	Group fitness classes
TrainingSessions	id (UUID)	trainer_id, member_id, room_id, date, times, status	Personal training sessions
Enrollments	id (UUID)	member_id, class_id, registered_at	Class enrolment records
TrainerAvailability	id (UUID)	trainer_id, date, start_at, end_at	Trainer schedule availability

2.4 Health and Billing Entities

Entity	Primary Key	Key Attributes	Purpose
FitnessGoals	id (UUID)	member_id, description, target_value	Member fitness objectives

Entity	Primary Key	Key Attributes	Purpose
HealthMetrics	id (UUID)	member_id, metric_type, value, recorded_at	Health tracking records
Subscriptions	id (UUID)	plan, fee	Membership plan definitions
Payments	id (UUID)	member_id, subscription_id, amount, status	Billing and payment records

2.5 Relationship Cardinality

The table below summarises the cardinality of all major relationships in the system. The two many-to-many relationships are resolved via junction tables to eliminate data redundancy and preserve referential integrity.

Relationship	Cardinality	Resolution
Users → Members / Trainers / AdminStaff	1 : 1	Shared primary key (FK inheritance)
Rooms → Equipments	1 : M	room_id FK on Equipments
Rooms → Classes / TrainingSessions	1 : M	room_id FK on each table
Trainers → Classes / Sessions / Availability	1 : M	trainer_id FK on each table
Members → FitnessGoals / HealthMetrics	1 : M	member_id FK on each table
Members ↔ Classes	M : N	Resolved via Enrollments junction
Members ↔ Trainers	M : N	Resolved via TrainingSessions junction

3. Relational Schema Design

3.1 Schema Mapping

Each entity identified in the ER model maps directly to a relational table. Primary keys are implemented as UUIDs throughout the schema to ensure global uniqueness and to support future distributed deployment. Foreign keys are declared explicitly with appropriate ON DELETE behaviour to protect referential integrity at the database engine level.

3.2 Key Design Decisions

(1) Inheritance Strategy. The Members, Trainers and AdminStaff tables share the same primary key value as the parent Users table (Members.id = Users.id). This one-to-one pattern ensures a user can occupy exactly one role and avoids duplicating authentication credentials across tables.

(2) Junction Tables. The Enrollments table resolves the many-to-many relationship between Members and Classes. A composite unique constraint on (member_id, class_id) prevents a member from enrolling in the same class twice.

(3) Constraint Enforcement. Business rules are pushed into the database layer using NOT NULL, UNIQUE, CHECK and DEFAULT constraints, reducing reliance on application-level validation alone.

(4) Audit Timestamps. All tables carry created_at and updated_at columns maintained automatically by triggers, supporting audit trails.

(5) Custom Enum Types. Six PostgreSQL ENUM types (user_role, session_status, equipment_status, and others) constrain status columns to a defined set of valid values.

3.3 Constraints Summary

Constraint Type	Approximate Count	Example
PRIMARY KEY	15	users.id UUID
FOREIGN KEY	20+	members.id REFERENCES users(id)
UNIQUE	8	(member_id, class_id) on enrollments

Constraint Type	Approximate Count	Example
CHECK	10+	end_time > start_time on classes
NOT NULL	50+	users.email, users.password
DEFAULT	15+	created_at DEFAULT now()

4. Normalization to 3NF

4.1 First Normal Form (1NF)

All tables satisfy the First Normal Form. Every column holds only atomic, indivisible values. No repeating groups are present, and a primary key is defined for every table. As an illustrative example, the HealthMetrics table stores each measurement as a separate row (e.g. weight and heart rate are distinct entries rather than a comma-delimited list within a single column).

4.2 Second Normal Form (2NF)

All tables satisfy the Second Normal Form. Because every table uses a single-column UUID primary key, partial dependencies are structurally impossible. Every non-key attribute is fully functionally dependent on that single key. The Classes table provides a clear example: the class name, date, start time, end time, and maximum capacity all depend solely on class_id and not on any subset of a composite key.

4.3 Third Normal Form (3NF)

All tables satisfy the Third Normal Form. There are no transitive dependencies anywhere in the schema: every non-key attribute depends on the primary key alone, and not on any other non-key attribute. The three most significant design decisions that enforce 3NF are described below.

(1) Users / Members separation. Authentication data (email, hashed password, role) resides in the Users table while profile data (name, phone, date of birth) resides in the Members table. Embedding email inside Members would create a transitive dependency Members.phone → Members.email → Users.id, violating 3NF.

(2) HealthMetrics isolation. Health data is stored in a dedicated table rather than as columns on the Members table. This prevents update anomalies when a member accumulates multiple health records over time.

(3) Subscriptions / MemberSubscriptions separation. Plan details (fee, description) reside in the Subscriptions table. The member's assignment to a plan is recorded in a separate MemberSubscriptions table, avoiding repetition of plan data on every

member row.

4.4 3NF Evidence Summary

Table	Non-Key Attributes	Functional Dependency	3NF Status
Users	email, password, role	Depend on id only	Satisfied
Members	full_name, phone, dob	Depend on id (not email)	Satisfied
Classes	name, date, times, capacity	Depend on class_id only	Satisfied
Equipments	name, status, notes	Depend on equipment_id	Satisfied
HealthMetrics	metric_type, value, recorded_at	Depend on id only	Satisfied
Enrollments	registered_at	Depends on id only	Satisfied

5. Database Implementation

5.1 DDL Overview

The DDL.sql file (481 lines) creates the complete database schema. The table below summarises all database objects created.

Object	Count	Notes
Tables	15	Full constraint coverage on all tables
Enum Types	6	user_role, session_status, equipment_status, and three others
Indexes	28	10 implicit PK indexes + 18 explicit performance indexes
Triggers	12	10 updated_at maintenance triggers + 2 business-rule triggers
Views	3	member_dashboard_view, trainer_schedule_view, equipment_maintenance_view

5.2 Database Views

View 1 — member_dashboard_view

This view combines data from six tables (Members, Users, HealthMetrics, FitnessGoals, Enrollments, TrainingSessions, and Trainers) into a single flat projection for the member-facing dashboard. It uses COUNT(DISTINCT) to aggregate the total number of classes attended per member and filters out soft-deleted records through a WHERE deleted_at IS NULL predicate.

View 2 — trainer_schedule_view

This view uses a UNION ALL to merge personal training sessions with group classes into a single, unified chronological schedule for each trainer. Only future-dated records are included (session_date ≥ CURRENT_DATE), ensuring the view always reflects upcoming commitments.

View 3 — equipment_maintenance_view

This view provides a human-readable maintenance status via a CASE expression that maps equipment status codes (under_repair, out_of_service, operational) to descriptive

labels. Results are ordered by status and room name to support the admin maintenance panel.

5.3 Business-Rule Triggers

Two triggers enforce complex business rules that would be fragile or cumbersome to enforce solely at the application level:

Trigger 1 — prevent_overlapping_bookings

This trigger fires BEFORE INSERT OR UPDATE on the training_sessions table. It evaluates time-range overlaps using three logical conditions simultaneously: overlap from the left, overlap from the right, and full containment. These checks are applied independently for the member, the trainer, and the room. If any conflict is detected, a PostgreSQL exception is raised and the transaction is rolled back, ensuring no double-bookings can exist in the database.

Trigger 2 — prevent_class_overbooking

This trigger fires BEFORE INSERT on the enrollments table. It counts current enrolments for the requested class and compares the count against the class's max_capacity attribute. If the class is already at capacity, an exception is raised, preventing enrolment regardless of the calling application or interface.

5.4 Performance Indexes

Eighteen explicit indexes are created on columns identified as frequent targets of filter predicates and join conditions:

Table	Indexed Column(s)	Query Pattern Supported
health_metrics	member_id	Member health history lookups
health_metrics	recorded_at	Time-range health queries
training_sessions	trainer_id	Trainer schedule queries
training_sessions	session_date	Date-range session queries
classes	room_id	Room availability checks

Table	Indexed Column(s)	Query Pattern Supported
classes	class_date	Schedule listing queries
equipments	status	Maintenance dashboard filter
enrollments	member_id, class_id (composite)	Enrolment lookup and duplicate check

6. Application Implementation

6.1 Backend Architecture

The backend follows a layered architecture with clear separation between HTTP routing, business logic, ORM models, and core infrastructure. Asynchronous database access via SQLAlchemy 2.0 ensures non-blocking I/O under concurrent load. The entire application — backend, database, and frontend — is containerised using Docker and orchestrated with Docker Compose, enabling consistent, reproducible deployments across environments with a single command.

Package	Contents and Responsibilities
core/	auth.py (JWT validation, RoleChecker), config.py, db.py (async pool), response.py
models/	SQLAlchemy ORM model classes for all 15 database tables
routes/	auth.py (login/logout), members.py (4 ops), trainers.py (2 ops), admin.py (2 ops)
services/	Business logic decoupled from the HTTP routing layer
migrations/	Versioned database migration scripts for schema management
Docker	Dockerfile (backend), Dockerfile (frontend), docker-compose.yml (orchestration)

6.2 API Endpoints

Member Operations (4)

Operation	Method	Endpoint	SQL Action
User Registration	POST	/members/register	INSERT INTO users, members
Profile Management	PUT	/members/me	UPDATE members SET ...
Health History	GET	/members/health-history	SELECT FROM health_metrics
Dashboard	GET	/members/dashboard	SELECT via member_dashboard_view

Trainer Operations (2)

Operation	Method	Endpoint	SQL Action
Set Availability	POST	/trainers/availability	INSERT INTO trainer_availability
Schedule View	GET	/trainers/schedule	SELECT via trainer_schedule_view

Admin Operations (2)

Operation	Method	Endpoint	SQL Action
Room Booking	PUT	/admin/sessions/{id}/room	UPDATE training_sessions
Equipment Maintenance	GET/PUT	/admin/equipment	SELECT / UPDATE equipments

6.3 Role-Based Access Control

Access control is implemented in core/auth.py through a reusable RoleChecker dependency class. On every request the JWT token is validated and the user's role is compared against the set of roles permitted for that endpoint. An HTTP 403 response is returned immediately on any mismatch. The access matrix below defines permissions for all eight operations:

Operation	Member	Trainer	Admin
User Registration	Permitted	Not Permitted	Permitted
Profile Management	Own record only	Not Permitted	Not Permitted
Health History	Own records	Assigned members	Not Permitted
Dashboard	Own data only	Not Permitted	Not Permitted
Set Availability	Not Permitted	Own record only	Not Permitted
Schedule View	Not Permitted	Own schedule	Permitted
Room Booking	Not Permitted	Not Permitted	Permitted
Equipment Maintenance	Not Permitted	Not Permitted	Permitted

6.4 Frontend Structure

The React and TypeScript frontend implements role-specific routing so that users only have access to pages appropriate for their assigned role. Each of the eight required operations is served by a dedicated page component:

Directory	Page Component(s)	Operations
pages/member/	RegistrationPage, ProfilePage, HealthHistoryPage, DashboardPage	1 – 4
pages/trainer/	AvailabilityPage, SchedulePage	5 – 6
pages/admin/	RoomBookingPage, EquipmentPage	7 – 8

- Role-specific routing with protected route guards
- Client-side form validation with descriptive error messages
- Loading states and skeleton screens for asynchronous data fetches
- Toast notifications for success and error feedback
- Pagination for all list-based views
- Modal dialogs for confirmations of destructive actions

7. Demonstration Video

7.1 Video Link

The full demonstration video is available at the link below. The video is approximately 12 minutes in duration, within the 15-minute limit.

Demonstration Video: [Insert YouTube / Google Drive URL here]

7.2 Video Content Outline

The demonstration video is structured into five sections as outlined below:

Segment	Duration	Content Covered
ER Model	~2 min	ER diagram walkthrough; entities, relationships, and cardinality
Relational Schema	~2 min	Schema diagram; table structures; primary and foreign keys
Database Implementation	~2 min	DDL.sql; views, triggers, and indexes; database creation
Operations Demonstration	~5 min	All 8 operations shown with both success and error scenarios
Role-Based Access Control	~1 min	Login with different roles; access restrictions demonstrated

8. Conclusion

8.1 Summary

This project successfully delivers a complete Health and Fitness Club Management System that satisfies every specified requirement. The implementation spans the full development lifecycle, from conceptual ER modelling and normalization analysis through to a fully operational full-stack web application enforcing role-based access control across three distinct user types.

Requirement	Status
All 8 operations implemented (4 Member + 2 Trainer + 2 Admin)	**Met**
15 entities with 15+ relationships in ER model	**Met**
3 database views covering all major dashboard needs	**Met**
12 triggers (10 timestamp + 2 business-rule)	**Met**
28 indexes (18 explicit performance indexes)	**Met**
Role-based access control for all 3 user types	**Met**
Third Normal Form throughout the relational schema	**Met**
PostgreSQL used as the primary database system	**Met**

8.2 Challenges Encountered

(1) Overlapping Time Logic. Implementing conflict detection for training session bookings required careful SQL logic to handle all three time-range overlap cases — overlap from the left, from the right, and full containment — within a single trigger function that checks the member, trainer, and room simultaneously.

(2) Many-to-Many Relationship Design. Designing the Enrollments and TrainingSessions junction tables while maintaining full referential integrity required composite unique constraints and careful consideration of cascading delete behaviour.

(3) Role-Based Routing Synchronisation. Ensuring that frontend route guards and backend RoleChecker dependencies remain consistent across all eight operations

required a shared role taxonomy enforced by the Users.role ENUM type at the database level.

8.3 Future Enhancements

The following enhancements could extend the system beyond the current project scope:

- Payment gateway integration (Stripe or PayPal) for online membership payments
- Email and push notifications for class reminders and session confirmations
- Mobile application (React Native) reusing the existing REST API layer
- Analytics dashboard featuring attendance trends and revenue charts
- Member attendance tracking via QR code scanning at room entry points

---

Appendix A — Default Test Credentials

Role	Email	Password
Admin	admin@gym.com	password123
Trainer	trainer1@gym.com	password123
Member	member1@gym.com	password123

— End of Report —